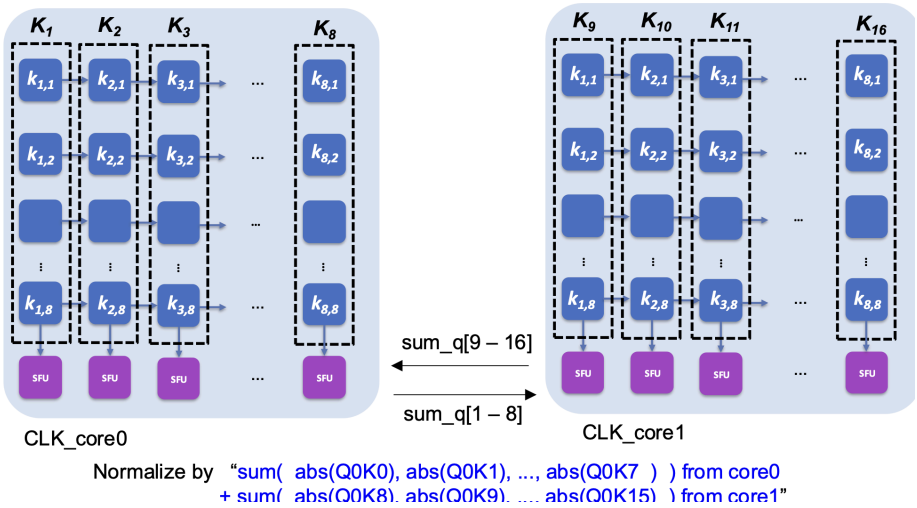
Dualcore Hardware Accelerator for Attention Mechanism in Machine Learning

Introduction

This project implements a dual-core machine learning accelerator for the attention mechanism. Each core is given two 8-element vectors of 8-bit integers and computes an 8x8 matrix of dot products. These matrices are summed, and each row of the combined matrix is then normalized to produce the output.



Step 1: Single Core RTL, Synthesis, and PNR

First, each core is given two 8-element vectors of 8-bit integers, Q and K. Their matrix product is computed to produce an 8x8 matrix of dot products
RTL skeleton code mostly worked. We added an output port attached to pmem_out for verification. There was also a bug in the skeleton code that set pr=16, when in reality, the dot products would only have pr=8; this required a modification to mac_16in.v as well to reduce down to 8 products.

We also got a testbench for norm*V which is similar matrix multiplication style as q*k.
Gate level sim VCD and terminal outputs screenshotted in Appendix.
Synthesis and PNR targeted 1 GHz, but did not meet timing, achieving a WNS of -0.510.

Step 2: Output Normalization

We added sfp_row into the core.

Normalization is performed in sfp_row.v in two phases: accumulation (acc) and division (div). During accumulation, the absolute values of eight elements are summed and stored in a FIFO. During division, each element is divided by the sum of both cores (sum_2core), which is computed combinationally from the output of the FIFO and the input of the other core (sum_in). We placed the sfp_row module which performs normalization in the core module, and its acc and div signals were controlled through the test bench. To ensure, the quotient was visible after dividing, the denominator i.e. the sum was right-shifted by seven bits.
Behavioral sim vcd and terminal output in Appendix

Step 3: Hierarchical Synthesis of Single Core

Hierarchical synthesis followed the lecture guidelines. For sram, we had to do synthesis and pnr for a 64b wide sram (q_mem and k_mem) and a 160b wide sram (psum_mem). We used a 4 micron output pin pitch as suggested. And because these srams were many bits wide and only a couple words tall, we used a wide rectangular shape for the floorplanning.

For the synthesis of core, we made sure to exclude the sram verilog files. For core pnr, we made it so that the psum mem was located towards the bottom of the core, where the output pins were. The qmem and kmem instances were rotated 90 degrees and placed along the left edge of the core.

WNS is -2.165 ns. Gate level sim passes as seen in the terminal output screenshot.

Step 4: Hierarchical Synthesis of Dual Core

To exchange the sums of the cores with each other, we used 2 asynchronous FIFOs: one to send data from Core 0 to Core 1 and another to send data from Core 1 to Core 0.
These async FIFOs are based on what was presented in lecture, with gray/binary converters and synchronizers between write and read domains in order to minimize metastability issues.

We also migrated the normalization control from the testbench into the fullchip.v file. Each core cycles through a finite state machine controller with READ, DIVIDE, and WRITE states. READ stage reads the row from psum mem, then moves to the DIVIDE stage which does the division, and then moves to the WRITE stage, which writes back to psum mem. Then it cycles back to READ. Since it's async, the READ stage doesn't move to the DIVIDE stage if it sees that the async fifo is empty. WNS: -1.44.

Step 5: Optimization

One optimizing technique we used was to pipeline the mac_col. In mac_16in.v, we registered each of the products before doing the addition step. We then realized the 8-way addition logic depth was still a bottleneck for performance. So we pipelined that further (making two more "cuts"). In each pipeline stage we would add two products instead of 8. So the first pipeline stage would have 4 pairs of 2-way additions. The second stage would have 2 pairs of 2-way additions. Finally, we'd have a single pair of 2-ways additions for final dot product output.

Similarly, in the sfp_row.v file, we found that the normalizing total sum should also be pipelined (2 more cuts), where we would only do pair-wise addition instead of 8-way addition.

Another optimizing technique incorporated was to use a multicycle path to allow for more delay to check setup and holdup time to ensure we reached near 0 worst negative slack. The different paths for the divider were set with multicycle 5 for setup time and multicycle 4 for hold time.

As we mentioned in Step 4, we added a controller to the full chip for the normalization phase control, which we think also counts as a contribution for Step 5.
Also note that for all the pipelining and the multicycle path, the FSM controller from step 4 had to be adjusted appropriately so that the pipelined/multicycle paths would still get the correct control signal timing.

Implementing multicycle for these paths coupled with pipelining in mac_16 and in sfp_row sum allowed the single core to achieve 0 WNS worst case for synthesis and 0 WNS typical case after pnr. Check out the step5/core/pnr/core.typical.timing.rpt for evidence of 0 WNS typical case.

Full chip hierarchical synthesis we got 0 WNS. However, for full chip hierarchical pnr, we got -1.3 WNS for some reason, even though there were no deep logic paths in fullchip.v. We saw that apparently the clock tree gave very large latencies. We tried so many different approaches and each time, syn/pnr took several hours; we changed verilog and added more pipelining, we changed the floor plan, we tinkered with the sdc file, and so on. Unfortunately, we could not figure out why the top fullchip.v module added so much latency in clock tree compared to the single core (single core was meeting 0 WNS).

Nonetheless, we have 0 DRC errors for the dual core. Gate simulation completed with zero errors in 175 ns (see Appendix).

Update: On March 21 4pm, we realized that we could try using non-hierarchical synthesis/pnr in order to solve the strange clock tree issues at the fullchip top module hierarchical pnr. This wasn't clear in the project instructions/FAQ, which is why we only just realized this. However, this non-hierarchical synthesis/pnr will likely finish on March 22 sometime around 2am. Here is an addendum for potential partial credit for getting 0 WNS:
ADDENDUM:
https://docs.google.com/document/d/1OaZri9Q4735OW_jfBRtW3nYOtPeeFnElmOKvXAEJP3E/edit?usp=sharing

Step 6: Row-level Sparsity Aware Attention

After accumulating a row of the dual-core matrix product QK0 + QK1, we calculate its magnitude. If the magnitude is below a certain threshold, we skip normalization for that row, saving clock cycles and thus power.

To calculate the threshold, we performed software simulations to calculate the 30th percentile of row magnitudes, using randomly generated 8x8 matrices with integer values from -8 to 7, similar to the given data in qdata.txt and kdata.txt. Across thousands of randomly generated matrices, the median 30th percentile hovered around 4, so we set the threshold to 4.

Sparsity is checked combinationally in fullchip.v: if a row's magnitude is below our threshold of 4, we skip the DIVIDE and WRITE stages entirely. Thus, we had to update fullchip.v's normalization controller that we mentioned in step 4, to add a new state, SKIP. if the controller sees a low sum from the READ state, then instead of moving to DIVIDE, it goes to the SKIP state, after which it will return to the READ state.

We finished hierarchical synthesis/pnr. Note that only fullchip.v was modified, so we were able to use the core.gds2 from step 5 as a submodule. 0 DRC errors. However, as noted in step 5, hierarchical pnr with fullchip seems to have issues getting the timing to work (-1.14 WNS), so check addendum:
ADDENDUM for potential partial credit:
https://docs.google.com/document/d/1OaZri9Q4735OW_jfBRtW3nYOtPeeFnElmOKvXAEJP3E/edit?usp=sharing

Verification was performed by mirroring hardware computations in the testbench and performing computations thusly.

Testbench tracks actual and expected skipped cycles to show that the improvement is what we expect from the threshold. End of the testbench generates many random numbers and tests if the sums < threshold will results in 25 to 45% sparsity as we want/
Gate simulation showed 4 errors.

Performance Analysis

	Before Optimization	After Step 5	After Step 6
Internal	12.93158103	8.70667264	7.87930361
Switching	10.34312696	4.77453557	3.37725436
Leakage	6.86173375	7.15022819	7.17673409
Total	30.13644194	20.63143642	18.4339210

We saw major decreases in power in every aspect after step 5 optimization

With the sparsity-aware optimization, internal and switching power decreased significantly at the cost of a slight leakage power increase. Total power consumption decreased by over 2 mW.

Conclusion

In this project, we learned the basic structure of a matrix multiplier (1D) along with a normalizer. We found that division is an expensive operation that could be 3-5 cycles, requiring the notation of a multicycle path in timing constraints. We also saw the immense value of pipelining for unnecessarily long logic paths like the dot product. Using sparsity for skipping and integrating that into a controller was also an endeavor that we accomplished.
Possibly the biggest takeaway we got was the tradeoff between hierarchical vs. non-hierarchical. We saw the value in compartmentalizing different parts of the chip, but for fullchip hierarchical synthesis and pnr, we found that the tools failed to reconcile 0WNS, even though each single core had enough slack. Our hypothesis is that non-hierarchical synthesis and pnr should do the trick in reaching 0 WNS or at least somewhat closer; this will be expanded upon in the addendum for partial credit hopefully.

Appendix

Addendum

https://docs.google.com/document/d/1OaZri9Q4735OW_jfBRtW3nYOtPeeFnElmOKvXAEJP3E/edit?usp=sharing

Implementation

Root Folder		/home/linux/ieng6/students/112/j2zamora/ece260b/dualcore_attention
Step 1	RTL	/step1/syn/verilog
	pnr-completed .enc	/step1/pnr/route.enc
	waveform vcd	/step1/gate_sim/core_tb.vcd /step1/gate_sim/core_tb_normv.vcd
Step 2	RTL	/step2/verilog/
	waveform vcd	/step2/fullchip_tb.vcd
Step 3	RTL	/step3/core/syn/verilog/
		/step3/sram_160b_w16/syn/verilog/
		/step3/sram_64b_w16/syn/verilog/
	pnr-completed .enc	/step3/core/pnr/route.enc
		/step3/sram_160b_w16/pnr/route.enc
		/step3/sram_64b_w16/pnr/route.enc
	waveform vcd	/step3/core/gate_sim/fullchip_tb.vcd
Step 4	RTL	/step4/syn/verilog
	pnr-completed .enc	/step4/pnr/route.enc
	waveform vcd	/step4/gate_sim/fullchip_tb.vcd
		/step4/gate_sim/fullchip_tb_normv.vcd
Step 5	RTL	/step5/syn/verilog
	pnr-completed .enc	/step5/pnr/route.enc
	waveform vcd	/step5/gate_sim/fullchip_tb.vcd
Step 6	RTL	/step6/syn/verilog
	pnr-completed .enc	/step6/pnr/route.enc
	waveform vcd	/step6/gate_sim/fullchip_tb.vcd

Gate Simulation Terminal Output Screenshots

```
##### Q data txt reading #####
##### K data txt reading #####
##### Estimated multiplication result #####
prd @cycle 0: 00006fffa3000640000cfffa0fffe0fffea00003
prd @cycle 1: 000230000100023fffe0ffffc00002ffff00004b
prd @cycle 2: fffc6fffa50005400030fffde0000a0002cffff0
prd @cycle 3: 00005fffd000050fffe7fffabfffaffffeafffbd
prd @cycle 4: 00028ffff80004b00038fffd6fffdfffffd20000b
prd @cycle 5: 0004200040ffffd0002e000250005f00024ffff6
prd @cycle 6: 0006100075fffd60001d000260002bffffefffd3
prd @cycle 7: 00004fffef0001a0000b0000000008ffff40001e
##### Qmem writing  #####
##### Kmem writing #####
##### K data loading to processor #####
##### execute #####
##### move ofifo to pmem #####
##### rd from pmem #####
Memory read from PSUM add 0 00006fffa3000640000cfffa0fffe0fffea00003
Memory read from PSUM add 1 000230000100023fffe0ffffc00002ffff00004b
Memory read from PSUM add 2 fffc6fffa50005400030fffde0000a0002cffff0
Memory read from PSUM add 3 00005fffd000050fffe7fffabfffaffffeafffbd
Memory read from PSUM add 4 00028ffff80004b00038fffd6fffdfffffd20000b
Memory read from PSUM add 5 0004200040ffffd0002e000250005f00024ffff6
Memory read from PSUM add 6 0006100075fffd60001d000260002bffffefffd3
Memory read from PSUM add 7 00004fffef0001a0000b0000000008ffff40001e
Simulation complete via $finish(1) at time 100 NS + 0
./netlist/core_tb.v:391   #10 $finish;
xcelium>
```

Step 1: Gate simulation

```
##### Q data txt reading #####
##### K data txt reading #####
##### Estimated multiplication result #####
prd @cycle 0: 00003000220002f0001a00014000140001400014
prd @cycle 1: 000150000b00017000040001400014000140014
prd @cycle 2: 000340003c0001a0001c00004000040000400004
prd @cycle 3: 000500001f0002e0001c00004000040000400004
prd @cycle 4: 0000b00012000010000500020000200002000020
prd @cycle 5: 0000b00012000010000500020000200002000020
prd @cycle 6: 0000b00012000010000500020000200002000020
prd @cycle 7: 0000b00012000010000500020000200002000020
##### Qmem writing  #####
##### Kmem writing #####
##### K data loading to processor #####
##### execute #####
##### move ofifo to pmem #####
output normalization
finsihed accumulating
##### rd from pmem #####
Memory read from PSUM add 0 00003000220002f0001a00014000140001400014
Memory read from PSUM add 1 000150000b00017000040001400014000140014
Memory read from PSUM add 2 000340003c0001a0001c00004000040000400004
Memory read from PSUM add 3 000500001f0002e0001c00004000040000400004
Memory read from PSUM add 4 0000b00012000010000500020000200002000020
Memory read from PSUM add 5 0000b00012000010000500020000200002000020
Memory read from PSUM add 6 0000b00012000010000500020000200002000020
Memory read from PSUM add 7 0000b00012000010000500020000200002000020
Simulation complete via $finish(1) at time 133 NS + 0
./netlist/core_tb.v:473   #10 $finish;
```

Step 2: Gate simulation

```
##### Q data txt reading #####
##### K data core 0 txt reading #####
##### K data core 1 txt reading #####
##### Estimated multiplication result #####
prd @cycle 0: 1800012000320000400005000150001600017000010000100000b00004000040000200000d00009
prd @cycle 1: 2000060001600007000a000020000e00023000070000000000000060000f000020000300015
prd @cycle 2: 40001c000280000f000120000500009000030000700004000b000070000c000000001100010
prd @cycle 3: 1900002000000001e000040000300003000090000500000f00005000070000a00009000040000c
prd @cycle 4: 50000a000000000300007000070001300000c000060000c0000a000090001100000000000000b
prd @cycle 5: 120000700012000030000200014000180000300001000130000d000100000100006000020000
prd @cycle 6: 6000100001c0000b000060000c000090000600005000130000a000100000100001000400001
prd @cycle 7: 360000c000230000b0000b000370000100013000e000150001500013000150000500011000004
##### Qmem writing  #####
##### Kmem writing #####
##### K data loading to processor #####
##### execute #####
##### move ofifo to pmem #####
output normalization
finsihed accumulating
##### rd from pmem #####
match: Memory read from PSUM add 0 000180001200032000040000500015000160001700001000010000b00004000040000200000d00009
match: Memory read from PSUM add 1 000020000600016000070000a000020000e00023000070000000000000060000f000020000300015
match: Memory read from PSUM add 2 000040001c000280000f000120000500009000030000700004000b000070000c000000001100010
match: Memory read from PSUM add 3 000190000200000001e000040000300003000090000500000f00005000070000a00009000040000c
match: Memory read from PSUM add 4 000050000a000000000300007000070001300000c000060000c0000a000090001100000000000000b
match: Memory read from PSUM add 5 000120000700012000030000200014000180000300001000130000d000100000100006000020000
match: Memory read from PSUM add 6 000060001000001c0000b000060000c000090000600005000130000a000100000100001000400001
match: Memory read from PSUM add 7 000360000c000230000b0000b000370000100013000e000150001500013000150000500011000004
total number of errors:  0
Simulation complete via $finish(1) at time 164 NS + 0
./netlist/fullchip_tb.v:545   #10 $finish;
```

Step 3: Gate simulation

```
VCD info: dumpfile fullchip_tb_normv.vcd opened for output.
##### V data txt reading #####
##### N data core 0 txt reading #####
##### N data core 1 txt reading #####
##### Estimated multiplication result #####
prd @cycle 0: 0000200015000070000900010000c000150000a000080000e000010000300004000080001600007
prd @cycle 1: 000040000e00004000030000d0000d000040000d0000a0000e0000a000040000000000b0001200002
prd @cycle 2: 000040000b000080000800000a000100000500005000040000f0000b000010000500009000b00001
prd @cycle 3: 000060000b0000b000080000b0000a000070000700005000d0000a000020000700008000800002
prd @cycle 4: 0001f0000c0000000060000a000040000c000090000300003000c000100001c0000d0000500001
prd @cycle 5: 000060001200000b000060000c000060000300000a000080000e000090000600009000030000700002
prd @cycle 6: 000070001400006000120001500002000080000d000020000c000040000200005000030001100001
prd @cycle 7: 000080000a000090000800009000080000700008000080000c000070000400007000070000900001
##### Vmem writing  #####
##### Nmem writing #####
##### N data loading to processor #####
##### execute #####
##### move ofifo to pmem #####
output normalization
finsihed accumulating
##### rd from pmem #####
match: Memory read from PSUM add 0 000020001500007000090001000000c000150000a000080000e000010000300004000080001600007
match: Memory read from PSUM add 1 000040000e00004000030000d0000d000040000d0000a0000e0000a000040000000000b0001200002
match: Memory read from PSUM add 2 000040000b000080000800000a000100000500005000040000f0000b000010000500009000b00001
match: Memory read from PSUM add 3 000060000b0000b000080000b0000a000070000700005000d0000a000020000700008000800002
match: Memory read from PSUM add 4 0001f0000c0000000060000a000040000c000090000300003000c000100001c0000d0000500001
match: Memory read from PSUM add 5 000060001200000b000060000c000060000300000a000080000e000090000600009000030000700002
match: Memory read from PSUM add 6 000070001400006000120001500002000080000d000020000c000040000200005000030001100001
match: Memory read from PSUM add 7 000080000a000090000800009000080000700008000080000c000070000400007000070000900001
total number of errors:  0
./veriloq/fullchip_tb_normv.v:548: $finish called at 164000 (1ps)
```

*Step 4: Gate simulation (norm * V)*

```
run
data txt reading #####
data core 0 txt reading #####
data core 1 txt reading #####
timated multiplication result #####
le 0: 1800012000320000400005000150001600017000010000100000b00004000040000200000d00009
le 1: 2000060001600007000a000020000e00023000070000000000000060000f000020000300015
le 2: 40001c000280000f000120000500009000030000700004000b000070000c000000001100010
le 3: 1900002000000001e000040000300003000090000500000f00005000070000a00009000040000c
le 4: 50000a000000000300007000070001300000c000060000c0000a000090001100000000000000b
le 5: 120000700012000030000200014000180000300001000130000d000100000100006000020000
le 6: 6000100001c0000b000060000c000090000600005000130000a000100000100001000400001
le 7: 360000c000230000b0000b000370000100013000e000150001500013000150000500011000004
em writing  #####
em writing #####
data loading to processor #####
ecute #####
ve ofifo to pmem #####
ormalization
l accumulating
l from pmem #####
lemory read from PSUM add 0 000180001200032000040000500015000160001700001000010000b00004000040000200000d00009
lemory read from PSUM add 1 000020000600016000070000a000020000e00023000070000000000000060000f000020000300015
lemory read from PSUM add 2 000040001c000280000f000120000500009000030000700004000b000070000c000000001100010
lemory read from PSUM add 3 000190000200000001e000040000300003000090000500000f00005000070000a00009000040000c
lemory read from PSUM add 4 000050000a000000000300007000070001300000c000060000c0000a000090001100000000000000b
lemory read from PSUM add 5 000120000700012000030000200014000180000300001000130000d000100000100006000020000
lemory read from PSUM add 6 000060001000001c0000b000060000c000090000600005000130000a000100000100001000400001
lemory read from PSUM add 7 000360000c000230000b0000b000370000100013000e000150001500013000150000500011000004
umber of errors:  0
```

*Step 4: Gate simulation (q * k)*

```
xm>
xm> source /software/ECE/XCELIUM-2009/tools/xcelium/files/xmsimrc
xm> run
 Q data txt reading #####
 K data core 0 txt reading #####
 K data core 1 txt reading #####
 Estimated multiplication result #####
ycle 0: 18000120003200004000050001500016000170000100001000b0000400004000020000d00009
ycle 1: 20000600016000070000a000020000e000230000700000000000000060000f000020000300015
ycle 2: 40001c000280000f000120000500009000030000700004000b000070000c000000001100010
ycle 3: 190000200000001e000040000300003000090000500000f000050000700000a00009000040000c
ycle 4: 50000a0000000003000070000700013000c000060000c0000a000090001100000000000000b
ycle 5: 120000700012000030000200014000180000300001000130000d000100000100006000200009
ycle 6: 6000100001c0000b000060000c000090000600005000130000a000100000100001000400001
ycle 7: 360000c000230000b0000b000370000100013000e000150001500013000150000500011000004
 Qmem writing  #####
 Kmem writing #####
 K data loading to processor #####
 execute #####
 move ofifo to pmem #####
: normalization
ed accumulating
 rd from pmem #####
  Memory read from PSUM add 0 000180001200032000040000500015000160001700001000010000b000040000400002000d00009
  Memory read from PSUM add 1 000020000600016000070000a000020000e000230000700000000000000060000f000020000300015
  Memory read from PSUM add 2 000040001c000280000f000120000500009000030000700004000b000070000c000000001100010
  Memory read from PSUM add 3 000190000200000001e000040000300003000090000500000f000050000700000a00009000040000c
  Memory read from PSUM add 4 000050000a0000000003000070000700013000c000060000c0000a000090001100000000000000b
  Memory read from PSUM add 5 000120000700012000030000200014000180000300001000130000d000100000100006000200009
  Memory read from PSUM add 6 00006000100001c0000b000060000c000090000600005000130000a000100000100001000400001
  Memory read from PSUM add 7 000360000c000230000b0000b000370000100013000e000150001500013000150000500011000004
 number of errors:  0
tion complete via $finish(1) at time 175 NS + 0
/linux/ieng6/ECE260B_WI26_A00/k3liang/dualcore-attention/step5/gate_sim/netlist/fullchip_tb.v:552
xm>
```

Step 5: Gate simulation

```
xcelium>
xcelium> source /software/ECE/XCELIUM-2009/tools/xcelium/files/xmsimrc
xcelium> run
##### Q data txt reading #####
##### K data core 0 txt reading #####
##### K data core 1 txt reading #####
##### Computing expected results (sparsity-aware) #####
SPARSE @row0: sum_2core=2 < threshold=4 -> SKIP state, pmem holds 0
ACTIVE @row1: sum_2core=4 -> normalize, expected=000020000600016000070000a000020000e000230000700000000000000060000f000020000300015
SPARSE @row2: sum_2core=3 < threshold=4 -> SKIP state, pmem holds 0
ACTIVE @row3: sum_2core=7 -> normalize, expected=000190000200000001e000040000300003000090000500000f000050000700000a00009000040000c
ACTIVE @row4: sum_2core=4 -> normalize, expected=000050000a0000000003000070000700013000c000060000c0000a000090001100000000000000b
SPARSE @row5: sum_2core=3 < threshold=4 -> SKIP state, pmem holds 0
ACTIVE @row6: sum_2core=6 -> normalize, expected=00006000100001c0000b000060000c000090000600005000130000a000100000100001000400001
SPARSE @row7: sum_2core=1 < threshold=4 -> SKIP state, pmem holds 0
Cycles without sparsity : 56
Cycles with    sparsity : 36  (20 saved)
##### Qmem writing  #####
##### Kmem writing #####
##### K data loading to processor #####
##### execute #####
##### move ofifo to pmem #####
output normalization
finished accumulating
##### rd from pmem #####
SKIP-IGNORED row0: sparse row, output undefined (skipping check)
ACTIVE-ERROR row1: expected 000020000600016000070000a000020000e000230000700000000000000060000f000020000300015, got 0000afffe7fffa70001c0002bffff70003b0008ffffe3000030000000190003effff60000d00055
SKIP-IGNORED row2: sparse row, output undefined (skipping check)
ACTIVE-ERROR row3: expected 000190000200000001e000040000300003000090000500000f000050000700000a00009000040000c, got fff4efffff200000fff2dfffdf0001800015fffc0fffdcfff93fffd7fffcafffb90003f00022fffa8
ACTIVE-ERROR row4: expected 000050000a0000000003000070000700013000c000060000c0000a000090001100000000000000b, got fffeb0002afffffdffff2fffe40001c0004c000310001bfffce00028fffdb0004600003fffff0002d
SKIP-IGNORED row5: sparse row, output undefined (skipping check)
ACTIVE-ERROR row6: expected 00006000100001c0000b000060000c000090000600005000130000a000100000100001000400001, got 0002700063000acfffbefffd7fffb3fffc5fffda0001ffff8cfffc0fff9e0000a00007fffe8ffff7
SKIP-IGNORED row7: sparse row, output undefined (skipping check)
total number of errors:  4
```

Step 6: Gate simulation (multiplication)

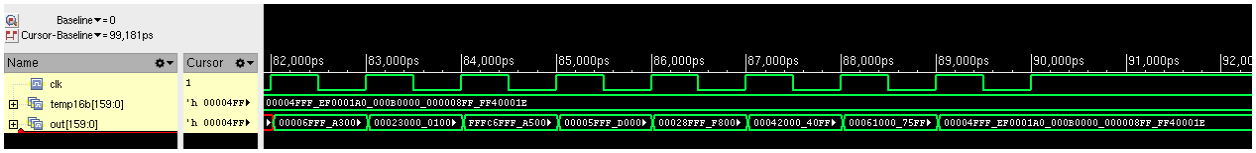
```
SKIP-IGNORED row5: sparse row, output undefined (skipping check)
ACTIVE-ERROR row6: expected 00006000100001c0000b000060000c0000900006000050001300000a0001
SKIP-IGNORED row7: sparse row, output undefined (skipping check)
total number of errors:  4

========================================
  Functional Verification Summary
  Total errors          : 4
========================================
  Skipping Report (THRESHOLD=4)
  Total rows            : 8
  Sparse rows skipped   : 4
  Active rows normalized: 4
  Sparsity rate         : 50%
  Cycles without skip   : 56
  Cycles with    skip   : 36
  Cycles saved          : 20  (5 per sparse row)
----------------------------------------
  Row 0: SKIPPED  sum_2core=2
  Row 1: ACTIVE   sum_2core=4
  Row 2: SKIPPED  sum_2core=3
  Row 3: ACTIVE   sum_2core=7
  Row 4: ACTIVE   sum_2core=4
  Row 5: SKIPPED  sum_2core=3
  Row 6: ACTIVE   sum_2core=6
  Row 7: SKIPPED  sum_2core=1
========================================

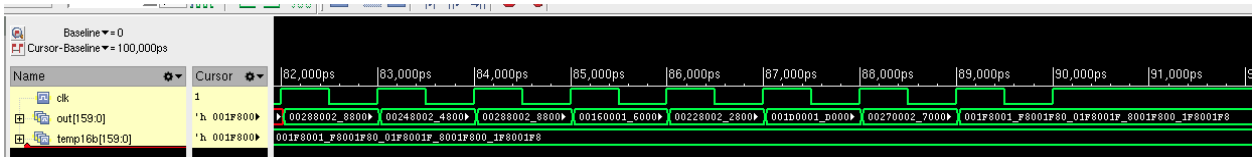
================================================
  30% Sparsity Verification (random data test)
  Threshold=4  Trials=1000
================================================
Average sparsity across all trials : 38.8%
Trials where >=3/8 rows sparse     : 634 / 1000 (63.4%)
PASS: average sparsity 38.8% is within expected range [25%, 45%]
================================================
Simulation complete via $finish(1) at time 175 NS + 0
./netlist/fullchip_tb.v:639   #10 $finish;
xcelium>
```

Step 6: Gate simulation (skipping)

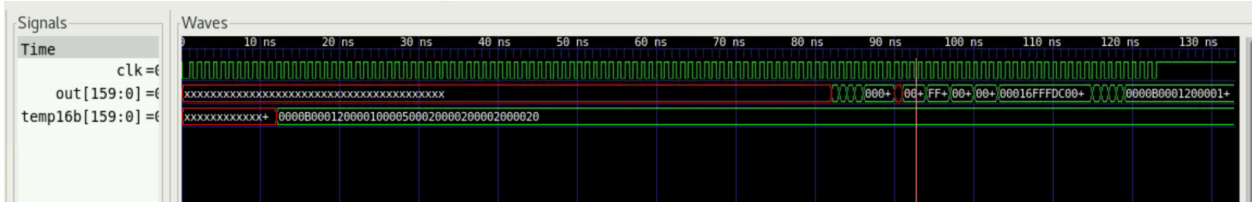
Waveforms for Steps 2-5



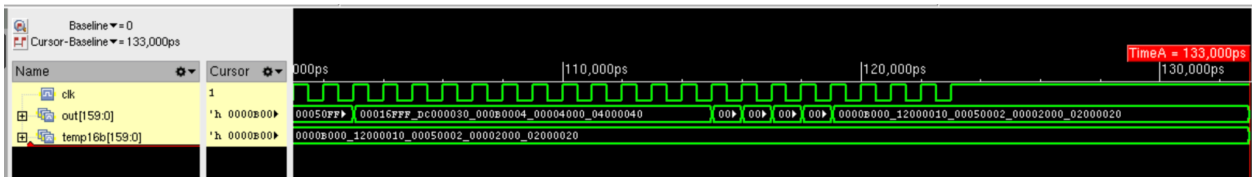
*Step 1: gate simulation waveform (q*k)*



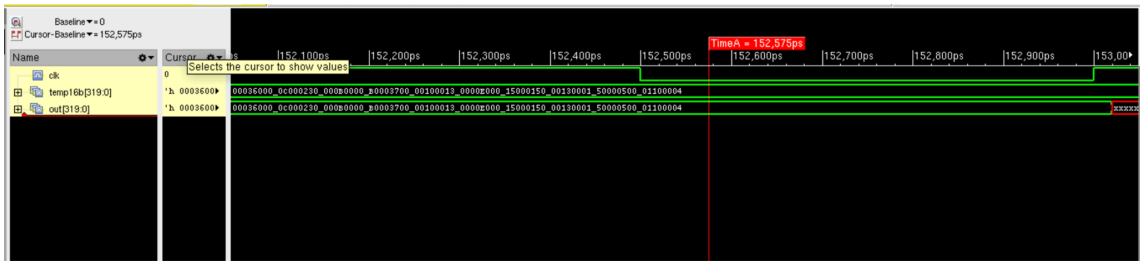
*Step 1: gate simulation waveform (norm*V)*



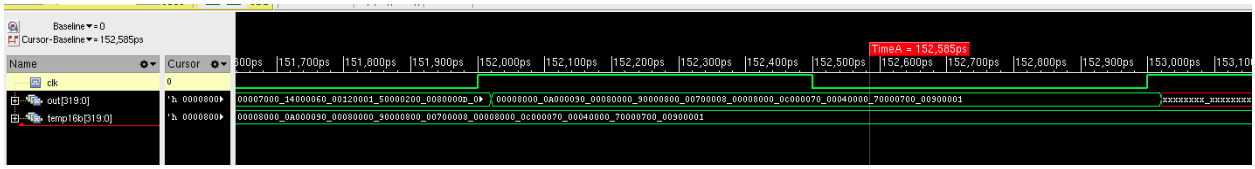
Step 2: gate simulation waveform



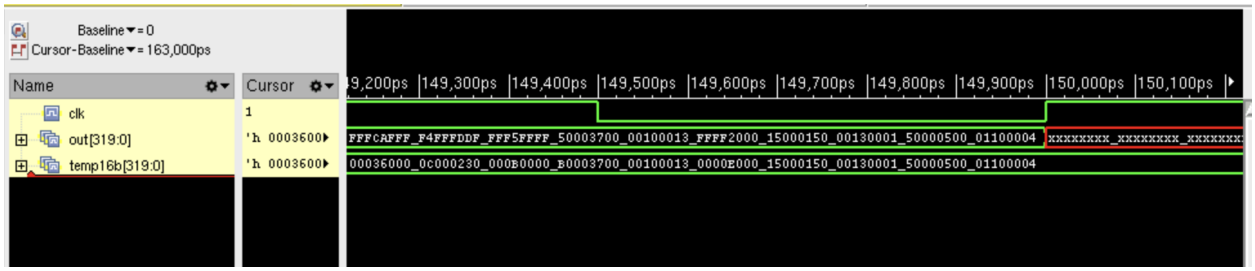
Step 3: Gate simulation waveform



*Step 4: gate simulation waveform (q*K)*



*Step 4: gate simulation waveform (norm*V)*



Step 5: gate simulation waveform



Step 6: gate simulation waveform

Power Reports for Steps 5 and 6

```
Total Power
--------------------------------------------------------------------
Total Internal Power:        8.70667264            42.2010%
Total Switching Power:       4.77453557            23.1420%
Total Leakage Power:         7.15022819            34.6570%
Total Power:                20.63143642
--------------------------------------------------------------------
Ended Static Power Report Generation: (cpu=0:00:00, real=0:00:00,
mem(process/total)=985.78MB/985.78MB)
```

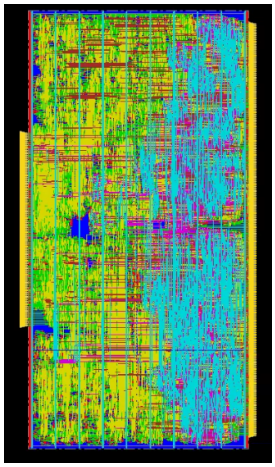
```
Total Power
--------------------------------------------------------------------
Total Internal Power:        7.87930361            42.7450%
Total Switching Power:       3.37725436            18.3215%
Total Leakage Power:         7.17673409            38.9335%
Total Power:                18.43329210
--------------------------------------------------------------------
Ended Static Power Report Generation: (cpu=0:00:00, real=0:00:00,
mem(process/total)=984.22MB/984.22MB)
```

Step 5 Addendum

FINISHED non-hierarchical PNR March 22 3:12am (took ~2 hours synthesis + ~7 hours PNR)

As mentioned, we already turned in a step 5 attempt on time, though it was using hierarchical synthesis and pnr.

But now, after running non-hierarchical synthesis and pnr (synthesizing all files at once), we get:
-0.007 WNS worst case
+0.225 slack typical case
Hold timing met best case
0 DRC
0 gate sim errors



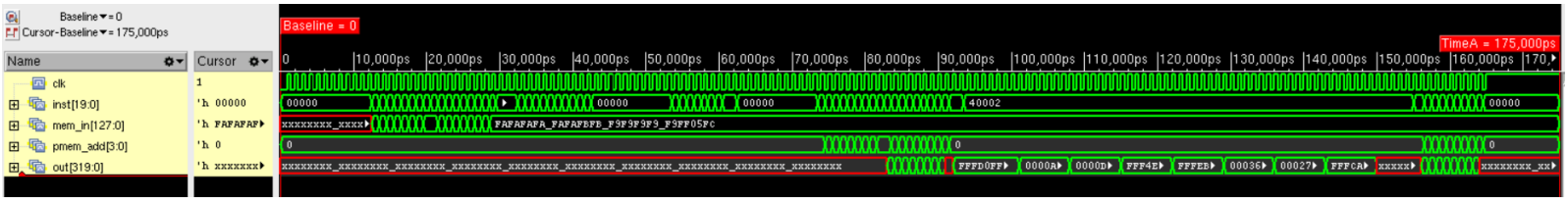
Working directory: /home/linux/ieng6/ECE260B_WI26_A00/k3liang/dualcore-attention

Step 5	RTL	/step5/syn2/verilog
	pnr-completed .enc	/step5/pnr2/route.enc
	waveform vcd	/step5/gate_sim/fullchip_tb.vcd

Gate-level sim terminal output

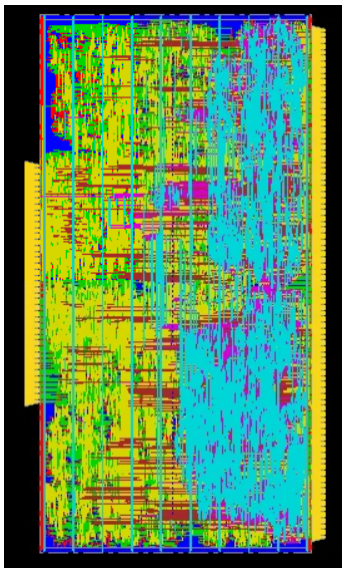
```
xcelium> run
##### Q data txt reading #####
##### K data core 0 txt reading #####
##### K data core 1 txt reading #####
##### Estimated multiplication result #####
prd @cycle 0: 18000120003200004000050001500016000170000100001000b000040000400002000d00009
prd @cycle 1: 2000060001600007000a000020000e000230000700000000000060000f000020000300015
prd @cycle 2: 40001c000280000f000120000500009000300007000040000b000070000c000000001100010
prd @cycle 3: 1900002000000001e000040000300003000090000500000f000050000700000a00009000040000c
prd @cycle 4: 50000a000000000300007000070001300000c000060000c0000a000090001100000000000000b
prd @cycle 5: 1200007000120000300002000140001800003000010001300000d000100000100006000200009
prd @cycle 6: 6000100001c0000b000060000c000090000600005000130000a000100000100001000400001
prd @cycle 7: 360000c000230000b0000b000370000100013000e000150001500013000150000500011000004
##### Qmem writing  #####
##### Kmem writing #####
##### K data loading to processor #####
##### execute #####
##### move ofifo to pmem #####
output normalization
finsihed accumulating
##### rd from pmem #####
match: Memory read from PSUM add 0 000180001200032000040000500015000160001700001000010000b000040000400002000d00009
match: Memory read from PSUM add 1 000020000600016000070000a000020000e000230000700000000000060000f000020000300015
match: Memory read from PSUM add 2 000040001c000280000f000120000500009000030000700004000b000070000c000000001100010
match: Memory read from PSUM add 3 000190000200000001e000040000300003000090000500000f000050000700000a00009000040000c
match: Memory read from PSUM add 4 000050000a000000000300007000070001300000c000060000c0000a000090001100000000000000b
match: Memory read from PSUM add 5 000120000700012000030000200014000180000300001000130000d000100000100006000200009
match: Memory read from PSUM add 6 00006000100001c0000b000060000c000090000600005000130000a000100000100001000400001
match: Memory read from PSUM add 7 000360000c000230000b0000b000370000100013000e000150001500013000150000500011000004
total number of errors:  0
Simulation complete via $finish(1) at time 175 NS + 0
./netlist/fullchip_tb.v:552    #10 $finish;
xcelium>
```

Gate-level sim VCD waveform



Step 6 Addendum

FINISHED non-hierarchical PNR March 22 5:38pm



We already turned in a hierarchical step 6 attempt on time, but with non-hierarchical synthesis/pnr, we get:

-0.005 WNS worst case
+0.253 slack typical case
Hold timing met best case
0 DRC
0 gate sim errors

Note that not only did we meet the typical case timing, but we also fixed it so that fullchip.pnr.v has 0 gate sim errors.

Working directory:
/home/linux/ieng6/ECE260B_WI26_A00/k3liang/dualcore-attention

Step 6	RTL	/step6/syn3/verilog
	pnr-completed .enc	/step6/pnr3/route.enc
	waveform vcd	/step6/gate_sim2/fullchip_tb.vcd

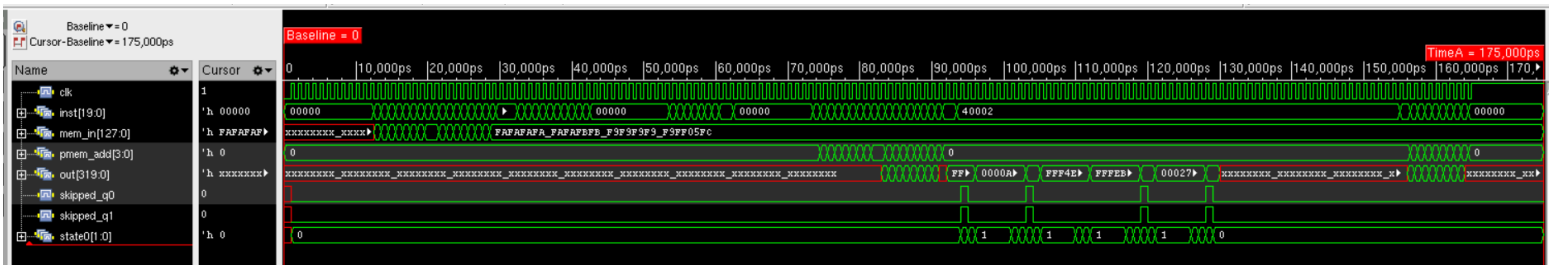
Gate-level sim terminal output

```
xcelium> run
##### Q data txt reading #####
##### K data core 0 txt reading #####
##### K data core 1 txt reading #####
##### Computing expected results (sparsity-aware) #####
SPARSE @row0: sum_2core=2 < threshold=4, expected=fffd0fffdcfff9bffff70000b0002b0002d0002f000030000200017ffff7ffff7ffffb0001bfffed
ACTIVE @row1: sum_2core=4 -> normalize, expected=000020000600016000070000a000020000e0002300007000000000000000060000f000020000300015
SPARSE @row2: sum_2core=3 < threshold=4, expected=0000dfffabfff860002d00037ffff10001c00009fffeafffff40002100015fffdcffffe00033fffcf
ACTIVE @row3: sum_2core=7 -> normalize, expected=000190000200000001e000040000300003000090000500001f00005000070000a00009000040000c
ACTIVE @row4: sum_2core=4 -> normalize, expected=000050000a000000000300007000070001300000c000060000c0000a000090001100000000000000b
SPARSE @row5: sum_2core=3 < threshold=4, expected=000360001500036000a00007fffc2fffb70000a00003fffc5fffd8fffceffffcfffec00008fffe5
ACTIVE @row6: sum_2core=6 -> normalize, expected=00006000100001c0000b000060000c000090000600005000130000a000100000100001000400001
SPARSE @row7: sum_2core=1 < threshold=4, expected=fffcaffff4fffddffff5ffff500037000010001 3ffff2000150001500013000150000500011000 04
Cycles without sparsity : 56
Cycles with    sparsity : 36  (20 saved)
##### Qmem writing  #####
##### Kmem writing #####
##### K data loading to processor #####
##### execute #####
##### move ofifo to pmem #####
output normalization
finished accumulating
##### rd from pmem #####
MATCH-SKIP row0: fffd0fffdcfff9bffff70000b0002b0002d0002f000030000200017ffff7ffff7ffffb0001bfffed
MATCH-ACTIVE row1: 000020000600016000070000a000020000e0002300007000000000000000060000f000020000300015
MATCH-SKIP row2: 0000dfffabfff860002d00037ffff10001c00009fffeafffff40002100015fffdcffffe00033fffcf
MATCH-ACTIVE row3: 000190000200000001e000040000300003000090000500001f00005000070000a00009000040000c
MATCH-ACTIVE row4: 000050000a000000000300007000070001300000c000060000c0000a000090001100000000000000b
MATCH-SKIP row5: 000360001500036000a00007fffc2fffb70000a00003fffc5fffd8fffceffffcfffec00008fffe5
MATCH-ACTIVE row6: 00006000100001c0000b000060000c000090000600005000130000a000100000100001000400001
MATCH-SKIP row7: fffcaffff4fffddffff5ffff500037000010001 3ffff2000150001500013000150000500011000 04
total number of errors:  0
```

```
==========================================
  Functional Verification Summary
  Total errors          : 0
==========================================
  Skipping Report (THRESHOLD=4)
  Total rows            : 8
  Sparse rows skipped   : 4
  Active rows normalized: 4
  Sparsity rate         : 50%
  Cycles without skip   : 56
  Cycles with    skip   : 36
  Cycles saved          : 20  (5 per sparse row)
------------------------------------------
  Row 0: SKIPPED  sum_2core=2
  Row 1: ACTIVE   sum_2core=4
  Row 2: SKIPPED  sum_2core=3
  Row 3: ACTIVE   sum_2core=7
  Row 4: ACTIVE   sum_2core=4
  Row 5: SKIPPED  sum_2core=3
  Row 6: ACTIVE   sum_2core=6
  Row 7: SKIPPED  sum_2core=1
==========================================

================================================
  30% Sparsity Verification (random data test)
  Threshold=4  Trials=1000
================================================
Average sparsity across all trials : 38.8%
Trials where >=3/8 rows sparse     : 634 / 1000 (63.4%)
PASS: average sparsity 38.8% is within expected range [25%, 45%]
================================================
Simulation complete via $finish(1) at time 175 NS + 0
./netlist/fullchip_tb.v:655   #10 $finish;
```

Gate-level sim VCD waveform



Updated Power/Performance Analysis

Using metrics for non-hierarchical approach for step 5 and step 6.

Step 4 Area was 2027643.200 um^2
Step 5 Area was 937848.000 um^2
Step 6 Area was 927587.400 um^2
The area reduction from step 4 to 5 is expected. Timing is impossible to meet in step 4, so cells get enlarged. Additionally, step 4 is with hierarchical synthesis/pnr, which also could cause waste of area.
The area reduction from step 5 to 6 is surprising; adding the skip hardware would be expected to increase the area of the chip, but it's possible that some sort of indirect pipelining/alternate placement in the skip hardware allowed timing to be more easily met and thus some cells shrunk.

As for performance, step 5 and 6 were able to meet setup timing for typical case and hold timing for best case. Step 5 had +0.225 typical case slack and Step 6 had +0.253 typical case slack. Surprisingly, it was easier to meet timing in Step 6, which is likely due to some logic path refactoring that happened for skip hardware.
Additionally, Step 5 and Step 6 were almost 0 WNS worst case. Compared to Step 4 with -1.441 WNS worst case, it's clear that the optimizations were critical for performance.

	Step 5	Step 6
Internal	357.62903146	348.19844350
Switching	249.33256662	235.72538923
Leakage	5.93718535	5.80787630
Total	612.89878447	589.73170989

About a 20mW power reduction from step 5 to step 6

ADDENDUM done.